MediMitra: How to Run the Project

Coworker Technical Setup & Operational Reference Guide

Rev 2.4 | Field Edition

DECISION SUPPORT ONLY. NOT A DIAGNOSIS.
MediMitra is an experimental technology prototype for community health worker decision support. It must never be used to establish a clinical diagnosis, advise drug dosages, or replace qualified medical officer evaluation. Use fictional cases only.

1. WHAT MEDIMITRA IS

MediMitra is an offline-first clinical triage decision support assistant tailored for community health workers (e.g. ASHA workers) operating in low-connectivity rural health posts. The application collects reported symptoms and vital measurements to assign an initial three-tier triage urgency: `GREEN` (Routine Follow-up), `YELLOW` (Priority PHC Assessment), or `RED` (Immediate Emergency Transfer). For every patient intake, it generates clarifying diagnostic questions, a plain-language clinical case summary, and a formal handoff referral note. In dual-evaluation mode, local on-device neural inference can be compared side-by-side with an escalated cloud second opinion. MediMitra is strictly an exploratory research prototype and must be evaluated using fictional cases only.

2. PREREQUISITES & WHAT TO INSTALL

Ensure the following software packages are installed on your workstation before starting setup:

Tool / Runtime	Required Version	Verification Command (Windows PowerShell)
Python	Python 3.11+ / 3.12	`python --version` (must be 3.11 or higher)
Node.js & npm	Node.js 18+ (LTS)	`node -v; npm -v`
Git	Git 2.x+	`git --version`
Ollama *(Optional)*	Latest	`ollama list` (Optional; only needed for local offline neural models)

3. FIRST-TIME SETUP (STEP-BY-STEP POWERSHELL COMMANDS)

Open **Windows PowerShell** and execute the following commands in sequence:

Step 1: Clone Repository & Navigate to Project Root

```
git clone <repository URL>
cd MediMitra
```

Step 2: Create and Activate Python Virtual Environment

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

PowerShell Execution Policy Tip: If script execution is blocked on your system, enable it for your current terminal session with: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` before activating the venv.

Step 3: Install Backend Dependencies

```
pip install -r backend/requirements.txt
```

Step 4: Install Frontend Node Dependencies

```
cd frontend
npm install
cd ..
```

Step 5: Copy Configuration Template

```
Copy-Item .env.example .env
```

By default, `.env.example` specifies `DEMO_MODE=1`. This allows the fullstack application to run immediately in simulated offline mode without any API keys, accounts, or external network connectivity.

Quick Start: Demo Mode & Live Evaluation

Standard Operating Procedure · Running & Testing

Decision support only. Not a diagnosis.

4. QUICK START IN DEMO MODE (DEFAULT: NO KEYS, NO INTERNET)

Demo mode runs completely offline using deterministic clinical heuristics. Open two PowerShell windows:

Terminal 1: Start Backend API Service (Port 8000)

```
.\.venv\Scripts\Activate.ps1
uvicorn app.main:app --app-dir backend --port 8000
```

Backend API starts at `http://localhost:8000` (Swagger docs at `http://localhost:8000/docs`).

Terminal 2: Start Frontend Application (Port 5173)

```
cd frontend
npm run dev
```

Access Application & Verify Status Header

- Open your browser at: `http://localhost:5173`
- **Check Top Header Status Pill:** It must display `Simulated demo`. This confirms that simulated mode is active.

5. CHECK THAT IT WORKS (VERIFY WITH BUILT-IN DEMOS)

Select and run these two built-in demo patients from the Home or Intake screen to verify correct operation:

- **Kabir Demo Case (`Kabir · fever and vomiting`):**
  8-year-old presenting with 3-day fever and vomiting twice (`temp: 39.0°C, pulse: 112 bpm`).
  Expected Urgency: `YELLOW` (Priority PHC medical assessment within 24 hours).
- **Dev Demo Case (`Dev · low oxygen rule override`):**
  47-year-old reporting feeling slightly unwell, but with critical hypoxemia (`SpO2 88%`).
  Expected Urgency: `RED` (Immediate emergency referral).
  Safety Override Card: A high-visibility warning card must appear stating:
  "Safety rules raised urgency to RED because: Oxygen saturation below 90% (88%)."
- **Statutory Disclaimer Check:** Verify that the exact text "*Decision support only. Not a diagnosis.*" is visible across every screen in the bottom footer and inside the printable referral note slip.

6. LIVE MODE (OPTIONAL: CLOUD SECOND-OPINION ESCALATION)

Live mode connects to high-parameter cloud neural models on Jarvislabs Token Factory for dual evaluation.

Step 1: Obtain Your Personal Jarvislabs Key

Every developer must use their **OWN** individual Jarvislabs API token. Never use a shared key.

Step 2: Configure Root `.env`

Variable	Recommended Value	Description
`DEMO_MODE`	`0`	Enables live neural model inference
`CLOUD_BASE_URL`	`https://models.jarvislabs.net/v1`	Jarvislabs Token Factory OpenAI-compatible URL
`JARVISLABS_API_KEY`	`<your_key_here>`	Your personal secret Jarvislabs API key
`CLOUD_MODEL`	`gemma-4-26b-a4b-it`	Served model ID on Jarvislabs catalog
`CLOUD_TIMEOUT_SECONDS`	`60`	Network timeout for cloud inference calls
`CLOUD_MAX_TOKENS`	`4000`	Output token limit preventing JSON truncation
`CLOUD_ENABLE_THINKING`	`0`	Disables thinking tags for clean JSON output

CRITICAL KEY SECURITY & PRIVACY RULES:

- **Server-Side Only:** Key goes *only* in root `.env`. NEVER put it in `frontend/.env`, any `VITE_` variable, Git, chat, or screenshots.
- **Restart Required:** Environment variables are loaded at startup. You **MUST restart the backend** (`Ctrl+C` and re-run uvicorn) whenever `.env` is modified.
- **Privacy Warning:** Cloud escalation sends case text to Jarvislabs. **Use fictional test cases only. Never enter real patient data.**

Troubleshooting & Incident Resolution

Common Symptoms, Root Causes & Verified Remediation Steps

Decision support only. Not a diagnosis.

7. TROUBLESHOOTING TABLE

Symptom	Likely Root Cause	Verified Remediation Step
"Status unavailable" in the top header	Frontend cannot reach backend `/api/status` endpoint (backend stopped or blocked on port 8000).	Ensure Terminal 1 is running: `uvicorn app.main:app --app-dir backend --port 8000`. Verify in browser at `http://localhost:8000/api/status`.
"Live: not configured" in the top header	`DEMO_MODE=0` is active, but cloud variables (`CLOUD_BASE_URL`, `JARVISLABS_API_KEY`, `CLOUD_MODEL`) or local models are missing.	Open root `.env` and fill in all three cloud configuration variables, or switch back to `DEMO_MODE=1`. Restart the backend process.
HTTP 401 Unauthorized from cloud escalation	The `JARVISLABS_API_KEY` in root `.env` is invalid, expired, or has insufficient credits.	Log in to your Jarvislabs console, generate a fresh API token, paste it into root `.env` as `JARVISLABS_API_KEY=<key>`, and restart the backend.
Model not found / 404 from API provider	The model identifier specified in `CLOUD_MODEL` or `LOCAL_MODEL` does not match the active server catalog.	Verify served model ID on Jarvislabs or check `ollama list`. Update `CLOUD_MODEL` in root `.env` to match the active catalog identifier.
"Deterministic Rules Active" / Rules-only banner displayed	AI model was unreachable, timed out, or output was rejected by the safety filter for containing dosage or medication advice.	Intentional clinical safety resilience. When model output is rejected or unavailable, the system safely falls back to deterministic rules.
HTTP 503 Service Unavailable on escalation	Cloud escalation credentials are unconfigured or the remote provider failed/timed out.	Verify internet connectivity and cloud credentials in `.env`. The frontend preserves and displays your local triage result regardless.
HTTP 429 Too Many Requests error	In-memory rate limiter exceeded (configured to 60 requests per minute per IP address).	Wait 60 seconds before submitting subsequent requests. Rate limits reset automatically on a rolling per-minute window.
Port already in use (Port 8000 or 5173 error)	A previous uvicorn or Vite dev server instance is still bound to the port in the background.	Find and terminate the process in PowerShell: `Get-NetTCPConnection -LocalPort 8000 | Select-Object OwningProcess` `Stop-Process -Id <pid>`
Virtual environment not activated (`uvicorn` / `pytest` command not found)	Commands executed in global Windows environment rather than inside virtual environment.	Run `.\.venv\Scripts\Activate.ps1` in your PowerShell terminal. Your prompt will display `(.venv)` when active.
`.env` **change not applied** after editing configuration	The backend server was not restarted after saving modifications to `.env`.	Environment variables are loaded strictly at Python interpreter startup. Press `Ctrl+C` in Terminal 1 and restart uvicorn.

Compliance, Safety Directives & Quick Reference

Safety Boundaries, Prohibited Actions & Command Cheat Sheet

Decision support only. Not a diagnosis.

8. MANDATORY PROJECT BOUNDARIES ("PLEASE DO NOT")

⚠ STRICT ETHICAL & SAFETY DIRECTIVES — PLEASE DO NOT:

1. **DO NOT Share or Commit Secrets:** Never commit `.env`, paste API keys into public chats, include keys in pull requests, or show keys in screenshots. All keys must remain strictly in your local, git-ignored root `.env`.
2. **DO NOT Put Keys in Frontend Code:** Never prefix API keys with `VITE_` or insert credentials into any file in `frontend/`. Vite bundles all `VITE_` variables into client-side JavaScript exposed publicly in the browser.
3. **DO NOT Enter Real Patient Data:** Never input real patient names, protected health information (PHI), or actual clinical records. Cloud escalation sends case narratives to external servers. Use only fictitious, synthetic test cases.
4. **DO NOT Use for Real Clinical Decision-Making:** MediMitra is an experimental technology prototype for research and software demonstration. It is not licensed medical software and must never be used to guide real-world patient care, establish medical diagnoses, or advise treatments and drug dosages.

9. QUICK REFERENCE COMMAND SUMMARY

Action	PowerShell Command
Activate Virtual Environment	`.\.venv\Scripts\Activate.ps1`
Run Backend Server (Port 8000)	`uvicorn app.main:app --app-dir backend --port 8000`
Run Frontend Server (Port 5173)	`cd frontend; npm run dev`
Run Full Test Suite (170 tests)	`pytest -q`
Verify Frontend Production Build	`cd frontend; npm run build`
Check Git Branch & Status	`git status; git branch`

10. ARCHITECTURE SUMMARY FOR COWORKERS

MediMitra's safety architecture consists of a deterministic rules engine (`backend/app/red_flags.py`) running strictly after AI model generation. Safety rules can only maintain or raise urgency levels, ensuring high-risk danger signs (e.g. SpO2 < 90%, chest pain, seizures, self-harm signals) force immediate red urgency regardless of LLM output. Model responses are filtered through an automated safety validator (`backend/app/output_filter.py`) that strictly rejects dosage amounts, administration verbs, or medication recommendations before the result reaches the health worker.

STATUTORY FIELD NOTICE: DECISION SUPPORT ONLY. NOT A DIAGNOSIS.

MediMitra is an experimental technology prototype. Always consult licensed medical officers and follow local health protocols.